

Dominando Web Push Notifications no Next.js

Guia Completo — Arquitetura, Segurança, UX e Produção

Service Worker

VAPID

Web Push API

Next.js 14+

TypeScript

2025 — Versão 2.0

Sumário

1. Introdução & Conceitos Fundamentais

- O que são Push Notifications?
- Por que usar no Next.js?
- Suporte a navegadores

2. Arquitetura Completa do Sistema

- Visão geral dos componentes
- Diagrama de fluxo
- Protocolo Web Push

3. Configuração do Ambiente

- Pré-requisitos
- Geração das chaves VAPID
- Variáveis de ambiente

4. O Service Worker (sw.js)

- Registro e ciclo de vida
- Evento push
- Evento notificationclick
- Estratégias de cache

5. Frontend — React Client Component

- Solicitando permissão
- Gerenciando assinaturas
- UX e estados de loading

6. Backend — Server Actions

- Configuração do web-push
- Persistência em banco de dados
- Segmentação de usuários

7. Segurança & Boas Práticas

- Chaves VAPID
- TTL e expiração
- Rate limiting

8. Notificações Avançadas

- Ações interativas
- Notificações silenciosas
- Push com imagem

9. Deploy & Produção

- Vercel, Railway e VPS
- Banco de dados em produção
- Monitoramento

10. Troubleshooting

- Erros comuns
- Debugging do SW
- Checklist final

1 Introdução & Conceitos Fundamentais

O que são Web Push Notifications?

Web Push Notifications são mensagens enviadas por um servidor diretamente para o navegador do usuário, mesmo quando o site não está aberto. Elas utilizam o protocolo **Web Push Protocol (RFC 8030)** combinado com a API de Service Workers para entregar mensagens em tempo real de forma confiável e segura.

O fluxo básico envolve três partes distintas: o **servidor de aplicação** (seu backend Next.js), o **servidor de Push** (operado pelo Google, Apple ou Mozilla) e o **navegador do usuário** com seu Service Worker registrado.

Dica

As Web Push Notifications são suportadas em Chrome, Firefox, Edge, Safari (iOS 16.4+) e Opera. São baseadas em padrões abertos — não dependem de SDKs proprietários.

Por que usar Push Notifications no Next.js?

- **Reengajamento:** Traga usuários de volta ao site sem precisar de um app nativo.
- **Tempo real:** Notifique sobre eventos críticos instantaneamente (pedidos, alertas).
- **Server Actions:** Next.js 14+ permite disparar notificações direto de ações do servidor.
- **App Router:** Compatível com a nova arquitetura de componentes do Next.js.
- **PWA:** Combinado com um manifest.json, transforma seu site em um Progressive Web App.
- **Sem push nativo:** Elimina a necessidade de React Native ou Flutter para alertas simples.

Comparativo: Push Web vs Push Nativo

Critério	Push Web	Push Nativo (iOS/Android)
Instalação de app	Não necessária	Obrigatória
Custo de infra	Baixo (FCM grátis)	Médio (APN \$ + FCM)
Permissão do usuário	Via browser popup	Via OS popup
Suporte offline	Limitado	Completo
Ações ricas	Até 2 ações	Múltiplas ações
Taxa de opt-in	~5-15%	~40-60%
Implementação	Web stack padrão	Código nativo / SDK

Suporte a Navegadores (2025)

Navegador	Versão Mínima	Suporte Push	Observações
Chrome / Chromium	42+	■ Completo	Usa FCM (Firebase)
Firefox	44+	■ Completo	Usa autopush.mozilla.org
Edge	17+	■ Completo	Usa WNS / FCM
Safari (macOS)	16+	■ Completo	Requer APNS key
Safari (iOS)	16.4+	■ PWA only	Apenas em home screen
Opera	29+	■ Completo	Usa FCM
IE / Samsung Internet	-	■ Sem suporte	Fallback necessário

2 Arquitetura Completa do Sistema

Visão Geral dos Componentes

A arquitetura de Web Push Notifications no Next.js é composta por quatro camadas bem definidas que trabalham em conjunto. Entender cada camada é fundamental para implementar um sistema robusto e escalável.

Camada	Tecnologia	Responsabilidade
Service Worker	sw.js (navegador)	Receber e exibir notificações em background
Frontend	React Client Component	Pedir permissão, gerenciar assinatura
Backend	Next.js Server Actions	Disparar notificações via web-push
Banco de Dados	PostgreSQL / MongoDB	Persistir assinaturas dos usuários
Push Server	FCM / APNs / Mozilla	Rotear notificações para o navegador correto

Diagrama de Fluxo Completo

1	Registro O site carrega e registra o Service Worker (sw.js) via <code>navigator.serviceWorker.register()</code>
2	Permissão O usuário clica em "Ativar notificações". O browser exibe o popup de permissão do sistema operacional.
3	Assinatura <code>PushManager.subscribe()</code> gera um objeto <code>PushSubscription</code> contendo endpoint único e chaves de criptografia.
4	Persistência O frontend envia o <code>PushSubscription</code> ao backend via Server Action. O backend salva no banco de dados vinculado ao usuário.
5	Disparo Quando um evento ocorre, o backend busca as assinaturas relevantes e chama <code>webpush.sendNotification()</code> .
6	Roteamento A biblioteca web-push envia um HTTP POST para o Push Server (FCM/APNs), que roteia para o dispositivo correto.
7	Recepção O Service Worker acorda via evento "push", processa os dados e chama <code>showNotification()</code> .
8	Interação O usuário vê a notificação. Ao clicar, o evento "notificationclick" é disparado e o site é aberto.

O Protocolo Web Push (RFC 8030)

O protocolo Web Push define como os servidores de aplicação se comunicam com os servidores de Push. Cada notificação é criptografada usando **ECDH (Elliptic Curve Diffie-Hellman)** com as chaves do objeto `PushSubscription`, garantindo que apenas o navegador do usuário consiga descriptografar o conteúdo.

■ Boas Práticas

A criptografia é feita em duas camadas:

1. Content-Encoding: aes128gcm (RFC 8291) — criptografa o payload
2. VAPID (RFC 8292) — autentica o servidor de origem

A biblioteca web-push cuida de toda essa complexidade automaticamente.

3 Configuração do Ambiente

Pré-requisitos

- Node.js 18.17+ (LTS recomendado)
- Next.js 14+ com App Router habilitado
- HTTPS obrigatório em produção (localhost funciona em desenvolvimento)
- Conta no banco de dados (PostgreSQL recomendado para produção)

Instalação das Dependências

■ Terminal — Instalação

```
# Instalar a biblioteca web-push
npm install web-push

# Tipos TypeScript (opcional mas recomendado)
npm install -D @types/web-push

# Para banco de dados (exemplo com Prisma + PostgreSQL)
npm install prisma @prisma/client
npx prisma init
```

Gerando as Chaves VAPID

As chaves VAPID (Voluntary Application Server Identification) são um par de chaves criptográficas que identificam seu servidor de aplicação. Elas garantem que apenas você possa enviar notificações para seus usuários. As chaves são geradas uma única vez e reutilizadas em toda a vida do projeto.

■ Geração de Chaves VAPID

```
# Método 1: Via CLI do web-push (recomendado)
npx web-push generate-vapid-keys

# Saída esperada:

# Public Key:
# BEl8BHjZCv5Nn4iMrGMJuNHbHRCL4Fhq5u5y...

# Private Key:
# Xt_jGdmh7Nm1cQbKFJEt_MsUyQYd3RHHQ...

# Método 2: Via código Node.js
```

```
const webpush = require("web-push");

const vapidKeys = webpush.generateVAPIDKeys();

console.log(vapidKeys.publicKey);

console.log(vapidKeys.privateKey);
```

■ ■ Atenção

NUNCA gere novas chaves VAPID em produção sem invalidar todas as assinaturas existentes!
Ao trocar as chaves, todos os usuários precisam se re-inscrever.
Armazene as chaves em um gerenciador de segredos (Vault, AWS Secrets Manager).

Configurando Variáveis de Ambiente

■ ■ .env.local — Variáveis de Ambiente

```
# .env.local

# Chave pública VAPID — prefixo NEXT_PUBLIC_ pois é usada no frontend
NEXT_PUBLIC_VAPID_PUBLIC_KEY=BE18BHjZCv5Nn4iMrGMJuNH...

# Chave privada VAPID — NUNCA prefixar com NEXT_PUBLIC_
VAPID_PRIVATE_KEY=Xt_jGdmh7Nm1cQbKFJEt_MsUy...

# Email de contato (obrigatório pelo protocolo VAPID)
VAPID_EMAIL=seu-email@dominio.com

# String de conexão com o banco de dados
DATABASE_URL=postgresql://user:pass@host:5432/pushdb
```

■ .gitignore — Protegendo Segredos

```
# .gitignore — certifique-se que .env.local está ignorado
.env.local
.env*.local

# Verifique com:
git check-ignore -v .env.local

# Saída esperada: .gitignore:N .env.local
```

4 O Service Worker (sw.js)

Ciclo de Vida do Service Worker

O Service Worker passa por três fases principais: **instalação** (install), **ativação** (activate) e **funcionamento** (fetch/push). Compreender esse ciclo é essencial para evitar bugs de cache e notificações duplicadas.

■ sw.js — Instalação e Ativação

```
// public/sw.js – Arquivo completo do Service Worker

// ■ Fase 1: Instalação ████████████████████████████████████████████████████

self.addEventListener("install", (event) => {
  console.log("[SW] Instalando...");

  // skipWaiting() faz o novo SW ativar imediatamente sem esperar
  // o antigo ser descarregado. Use com cuidado em produção.

  self.skipWaiting();
});

// ■ Fase 2: Ativação ████████████████████████████████████████████████████

self.addEventListener("activate", (event) => {
  console.log("[SW] Ativado. Controlando clientes...");

  // clients.claim() faz o SW assumir controle de todas as abas
  // abertas imediatamente, sem precisar recarregar a página.

  event.waitUntil(clients.claim());
});
```

■ sw.js — Recebendo e Exibindo Notificações

[illegible]

5 Frontend — React Client Component

Estrutura Completa do Componente

O componente de gerenciamento de Push Notifications deve ser robusto, lidar com todos os estados possíveis (loading, erro, não suportado, inscrito, não inscrito) e fornecer uma UX clara ao usuário.

■ components/PushNotificationManager.tsx — Parte 1

```
"use client";

import { useState, useEffect, useCallback } from "react";
import { subscribeUser, unsubscribeUser, sendNotification } from "@app/actions";

// Converte chave VAPID Base64URL para Uint8Array (requisito do PushManager)
function urlBase64ToUint8Array(base64String: string): Uint8Array {
  const padding = "=".repeat((4 - (base64String.length % 4)) % 4);
  const base64 = (base64String + padding)
    .replace(/-/g, "+")
    .replace(/_/g, "/");
  const rawData = window.atob(base64);
  return new Uint8Array([...rawData].map((c) => c.charCodeAt(0)));
}

type NotificationState = "idle" | "loading" | "subscribed" |
  "unsubscribed" | "denied" | "unsupported";

export default function PushNotificationManager() {
  const [state, setState] = useState<NotificationState>("idle");
  const [subscription, setSubscription] = useState<PushSubscription|null>(null);
  const [error, setError] = useState<string | null>(null);
  const [message, setMessage] = useState("");
```

■ components/PushNotificationManager.tsx — Parte 2 (useEffect)

```
// Verificar suporte e assinatura existente ao montar
useEffect(() => {
  const checkSupport = async () => {
    if (!("serviceWorker" in navigator) || !("PushManager" in window)) {
      setState("unsupported");
    }
    return;
  }
}
```

```
if (Notification.permission === "denied") {
  setState("denied");
  return;
}

const reg = await navigator.serviceWorker.register("/sw.js", {
  scope: "/",
  updateViaCache: "none",
});

const sub = await reg.pushManager.getSubscription();
if (sub) {
  setSubscription(sub);
  setState("subscribed");
} else {
  setState("unsubscribed");
}

};

checkSupport();

}, []);
```

■■ components/PushNotificationManager.tsx — Parte 3 (handlers)

```
const subscribeToPush = useCallback(async () => {
  setState("loading");
  setError(null);

  try {
    const reg = await navigator.serviceWorker.ready;
    const sub = await reg.pushManager.subscribe({
      userVisibleOnly: true,
      applicationServerKey: urlBase64ToUint8Array(
        process.env.NEXT_PUBLIC_VAPID_PUBLIC_KEY!
      ),
    });
    setSubscription(sub);
    setState("subscribed");
    await subscribeUser(JSON.parse(JSON.stringify(sub)));
  } catch (err: any) {
```

```
setError(err.message);  
setState("unsubscribed");  
}  
}, []);  
  
const unsubscribeFromPush = useCallback(async () => {  
  setState("loading");  
  await subscription?.unsubscribe();  
  setSubscription(null);  
  setState("unsubscribed");  
  await unsubscribeUser();  
}, [subscription]);
```

■ Boas Práticas

Use `useCallback()` em funções de efeito colateral para evitar re-renders desnecessários ao passar como props ou usar em `useEffect` com dependências.

6 Backend — Server Actions

Schema do Banco de Dados (Prisma)

Em produção, as assinaturas devem ser salvas em um banco de dados real, vinculadas ao usuário autenticado. Abaixo um schema Prisma completo para PostgreSQL.

■■ [prisma/schema.prisma](#) — Schema Completo

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url = env("DATABASE_URL")
}

model User {
  id String @id @default(cuid())
  email String @unique
  name String?
  subscriptions PushSubscription[]
  createdAt DateTime @default(now())
}

model PushSubscription {
  id String @id @default(cuid())
  userId String
  user User @relation(fields: [userId], references: [id])
  endpoint String @unique
  p256dh String // Chave pública de criptografia
  auth String // Segredo de autenticação
  userAgent String? // Identificação do browser
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  @@index([userId])
}
```

```
}
```

■■ app/actions.ts — subscribeUser com Banco de Dados

```
"use server";

import webpush from "web-push";

import { prisma } from "@lib/prisma";
import { auth } from "@lib/auth"; // seu sistema de autenticação

webpush.setVapidDetails(
  `mailto:${process.env.VAPID_EMAIL}`,
  process.env.NEXT_PUBLIC_VAPID_PUBLIC_KEY!,
  process.env.VAPID_PRIVATE_KEY!,
);

export async function subscribeUser(sub: any) {
  const session = await auth(); // NextAuth / Clerk / etc.
  if (!session?.user?.id) throw new Error("Não autenticado");

  await prisma.pushSubscription.upsert({
    where: { endpoint: sub.endpoint },
    create: {
      userId: session.user.id,
      endpoint: sub.endpoint,
      p256dh: sub.keys.p256dh,
      auth: sub.keys.auth,
      userAgent: sub.userAgent,
    },
    update: { updatedAt: new Date() },
  });

  return { success: true };
}
```

■ app/actions.ts — Enviando para Usuário com Limpeza Automática

```
export async function sendNotificationToUser(
  userId: string,
  payload: { title: string; body: string; url?: string; icon?: string }
) {
```

```
const subscriptions = await prisma.pushSubscription.findMany({
  where: { userId },
});

const results = await Promise.allSettled(
  subscriptions.map(async (sub) => {
    try {
      await webpush.sendNotification(
        { endpoint: sub.endpoint,
          keys: { p256dh: sub.p256dh, auth: sub.auth } },
        JSON.stringify(payload)
      );
    } catch (err: any) {
      // Assinatura expirada ou inválida – remover do banco
      if (err.statusCode === 410 || err.statusCode === 404) {
        await prisma.pushSubscription.delete({
          where: { id: sub.id },
        });
      }
      throw err;
    }
  })
);

return results;
}
```

7 Segurança & Boas Práticas

Chaves VAPID — Gerenciamento Seguro

- Gere as chaves VAPID **uma única vez** por domínio e armazene com segurança.
- Use um **gerenciador de segredos**: AWS Secrets Manager, HashiCorp Vault, Doppler.
- Nunca commite chaves no repositório — use **.env.local** e variáveis de CI/CD.
- Rotacione as chaves apenas em emergências (comprometimento). Invalida todos os subscribers.
- Monitore o **email VAPID** — o servidor de Push pode enviar alertas de abuso.

TTL (Time To Live) e Expiração

O TTL define por quanto tempo um servidor de Push deve tentar entregar uma notificação caso o dispositivo esteja offline. Configure-o com base na urgência do conteúdo.

Tipo de Notificação	TTL Recomendado	Exemplo
Tempo real / urgente	0 segundos	Alerta de fraude, autenticação 2FA
Transacional	3600 (1 hora)	Confirmação de pedido, pagamento
Engajamento	86400 (24h)	Newsletter, promoção do dia
Conteúdo evergreen	604800 (1 semana)	Novo artigo, feature update

■ Configurando TTL e Urgência

```
// Configurando TTL ao enviar notificação
await webpush.sendNotification(
  subscription,
  JSON.stringify(payload),
  {
    TTL: 3600, // 1 hora em segundos
    urgency: "normal", // "very-low" | "low" | "normal" | "high"
    topic: "orders", // Substitui notifs anteriores do mesmo tópico
  }
);
```

Rate Limiting — Evitando Spam

■ Rate Limiting com Upstash Redis

```
// lib/rateLimiter.ts – Usando Upstash Redis + ratelimit
import { Ratelimit } from "@upstash/ratelimit";
import { Redis } from "@upstash/redis";

const ratelimit = new Ratelimit({
  redis: Redis.fromEnv(),
  limiter: Ratelimit.slidingWindow(5, "1 h"), // 5 notifs/usuário/hora
  analytics: true,
});

export async function checkRateLimit(userId: string) {
  const { success, reset } = await ratelimit.limit(userId);
  if (!success) {
    const waitSeconds = Math.ceil((reset - Date.now()) / 1000);
    throw new Error(`Rate limit atingido. Tente em ${waitSeconds}s`);
  }
}
```

■ ■ Atenção

Nunca envie mais de 1 notificação por hora para o mesmo usuário em conteúdo de marketing. Para notificações críticas (2FA, alertas de fraude), sem limite. Monitore sua taxa de opt-out — acima de 5% indica abuso de frequência.

8 Notificações Avançadas

Notificações com Ações (Botões Interativos)

Notificações com ações permitem que o usuário interaja sem abrir o site, como confirmar um pedido ou marcar uma tarefa como concluída. Suportado em Chrome e Edge (até 2 ações por notificação).

■ Notificações com Botões de Ação

```
// No Service Worker: definindo as ações

const options = {
  body: "Seu pedido #4521 foi confirmado!",
  icon: "/icons/icon-192.png",
  actions: [
    {
      action: "view-order", // ID da ação
      title: "Ver Pedido", // Texto do botão
      icon: "/icons/bag.png", // Ícone do botão (opcional)
    },
    {
      action: "dismiss",
      title: "Dispensar",
      icon: "/icons/x.png",
    },
  ],
  data: { orderId: "4521", url: "/orders/4521" },
};

// Tratando o clique na ação específica
self.addEventListener("notificationclick", (event) => {
  event.notification.close();

  const { action, notification } = event;
  const { orderId, url } = notification.data;

  if (action === "view-order") {
    event.waitUntil(clients.openWindow(url));
  } else if (action === "dismiss") {
    // Apenas fecha – nenhuma ação adicional
  }
});
```

```
} else {  
  // Clique no corpo da notificação  
  event.waitUntil(clients.openWindow("/"));  
}  
});
```

Notificação com Imagem (Rich Push)

■ Rich Push — Notificação com Imagem Banner

```
// Backend: payload com imagem  
  
const payload = {  
  title: "Nova promoção: 50% OFF",  
  body: "Apenas hoje! Corra antes que acabe.",  
  icon: "/icons/icon-192.png",  
  image: "https://cdn.seusite.com/banner-promo.jpg", // 2:1 ratio ideal  
  url: "/promocoes",  
  tag: "promo-50", // Substitui notificação anterior com mesmo tag  
};  
  
await webpush.sendNotification(subscription, JSON.stringify(payload));
```

Notificações Silenciosas (Data Push)

Notificações silenciosas entregam dados ao Service Worker sem mostrar nada ao usuário. Úteis para sincronização de dados em background.

■ Data Push — Sincronização Silenciosa

```
// Backend: enviar payload sem título/body visível  
  
const silentPayload = {  
  silent: true,  
  type: "sync-data",  
  data: { userId: "123", newMessages: 5 }  
};  
  
// Service Worker: tratar sem exibir notificação  
  
self.addEventListener("push", (event) => {  
  const data = event.data.json();
```

```
if (data.silent) {  
  // Sincronizar dados sem notificar o usuário  
  event.waitUntil(  
    syncDataInBackground(data.data)  
  );  
  return; // Não chama showNotification()  
}  
  
// Push normal com notificação visual  
event.waitUntil(  
  self.registration.showNotification(data.title, { body: data.body })  
);  
});
```

■ Evite

Atenção: O `userVisibleOnly: true` na assinatura OBRIGA que toda notificação recebida seja exibida ao usuário. Notificações verdadeiramente silenciosas só são possíveis com `userVisibleOnly: false`, que está desabilitado no Chrome.

9 Deploy & Produção

Deploy na Vercel

- Configure as variáveis de ambiente no dashboard da Vercel (Settings > Environment Variables).
- O `sw.js` deve estar na pasta **public/** para ser acessível em `/sw.js`.
- Certifique-se que o domínio usa **HTTPS** — a Vercel fornece automaticamente.
- Server Actions funcionam nativamente — nenhuma configuração extra necessária.

■■ next.config.js — Headers do Service Worker

```
// next.config.js — Configurações recomendadas para produção
const nextConfig = {
  headers: async () => [
    {
      source: "/sw.js",
      headers: [
        // Service Workers devem ter Cache-Control: no-cache
        // para que updates sejam aplicados rapidamente
        { key: "Cache-Control", value: "no-cache, no-store, must-revalidate" },
        { key: "Content-Type", value: "application/javascript; charset=utf-8" },
      ],
    },
  ],
};

module.exports = nextConfig;
```

Banco de Dados em Produção — PostgreSQL no Supabase

■■ Configurando Banco de Dados em Produção

```
# 1. Criar conta em supabase.com e criar projeto

# 2. Copiar a connection string do painel SQL
# DATABASE_URL=postgresql://postgres:[SENHA]@db.[ID].supabase.co:5432/postgres

# 3. Rodar as migrations
npx prisma migrate deploy
```

```
# 4. Gerar o client atualizado  
npx prisma generate  
  
# Alternativas ao Supabase:  
# - Neon (serverless PostgreSQL nativo para Vercel)  
# - PlanetScale (MySQL serverless)  
# - MongoDB Atlas (NoSQL)  
# - Railway (PostgreSQL completo com backups)
```

Monitoramento e Observabilidade

Métrica	Ferramenta	O que monitorar
Taxa de entrega	Logs do web-push	Status codes 201 (sucesso) vs 4xx/5xx
Assinaturas ativas	Dashboard próprio	Total, crescimento, churn diário
Erros de SW	Sentry / Datadog	Exceções no service worker
Taxa de cliques (CTR)	Analytics custom	Notificationclick events
Rate de opt-out	Eventos do browser	Usuários que bloquearam permissão

10

Troubleshooting & Checklist Final

Erros Comuns e Soluções

■ **DOMException: Registration failed**

O sw.js não está na pasta public/ ou contém erros de sintaxe. Solução: Abra DevTools > Application > Service Workers para ver o erro exato.

■ **NotAllowedError: permission denied**

O usuário bloqueou ou o site não tem HTTPS. Solução: Use HTTPS em produção. Em dev, localhost funciona sem HTTPS.

■ **Error: 410 Gone (Push Server)**

A assinatura expirou ou o usuário desinstalou o browser. Solução: Remova automaticamente assinaturas com status 410 do banco.

■ **Error: 401 Unauthorized (VAPID)**

Chaves VAPID incorretas ou mal formatadas. Solução: Regere as chaves e verifique se NEXT_PUBLIC_ está no prefixo correto.

■ **SW não atualiza após mudanças**

O navegador cacheia o Service Worker. Solução: Use updateViaCache: "none" no register() e Cache-Control: no-cache no header.

Checklist de Deploy em Produção

- ☐ sw.js está em public/sw.js e acessível via /sw.js
- ☐ HTTPS configurado no domínio de produção
- ☐ Variáveis VAPID configuradas no servidor (não commitar no git)
- ☐ Cache-Control: no-cache no header do /sw.js
- ☐ Banco de dados com tabela de assinaturas criada (migrations rodadas)
- ☐ Limpeza automática de assinaturas expiradas (status 410/404)
- ☐ Rate limiting ativado para evitar spam
- ☐ Testado em Chrome, Firefox e Safari (iOS 16.4+ em PWA)
- ☐ TTL configurado de acordo com urgência das notificações
- ☐ Logs e monitoramento de erros configurados (Sentry/Datadog)

[] Permissão solicitada em contexto (não na entrada do site)

[] Fallback para usuários sem suporte a Service Workers

■ Boas Práticas

Dica final: Solicite permissão de notificação APENAS após o usuário realizar uma ação que demonstre intenção (ex: clicar em "Ativar notificações"). Popups imediatos na entrada do site têm taxa de aceitação de apenas 3-5%.

Contexto correto = 30-50% de aceitação!